Sobey School of Business
Saint Mary's University

Week 11

Introduction to Markov Chains

Instructor: Majid Taghavi (*majid.taghavi@smu.ca*)

MBAN 5540
Prescriptive Analytics and Optimization

Introduction

Markov Chains (MCs) are a powerful tool in both theoretical mathematics and practical applications because they enable the modeling of systems that change over time.

Here are a few key reasons why they are so important:

1. Predictive Modeling in Uncertain Systems
2. Powering Modern Search (PageRank)
3. Natural Language Processing (NLP)
4. Queueing Theory and Operations
5. Monte Carlo Simulations

Stochastic Process

Notation

- Let $X(t)$ denote the **state of a system** at time t.
- A **stochastic process** is a collection of random variables $X = \{X(t), t \in T\}$, where T is called the **index set**.

Example: IBM Stock Price

- $X(t)$ can represent the closing price of an IBM stock on day t.

Classifying the Index Set T

The nature of the index set T determines the type of process:

Discrete-Time Process

If the index set T is **countable** (e.g., $T = \{0, 1, 2, \dots\}$), X is a discrete-time stochastic process.

- *Example:* Daily closing prices of a stock.

Continuous-Time Process

If T is an **interval of the real line**, X is a continuous-time stochastic process.

- *Example:* The price of a stock at any time t throughout a trading day.

The State Space S

The set S of all possible values that $X(t)$ can take is called the **state space**.

- Supplier Reliability: $S = \{$Reliable, Delayed, Disrupted$\}$.
- Inventory Level: $S = \{0, 1, 2, \ldots, K\}$, where K is the maximum warehouse capacity.
- Health Status: $S = \{$Critical, Stable, Recovering$\}$.
- Operational Mode: $S = \{$Idle, Setup, Running, Maintenance, Breakdown$\}$.
- Market Trend: $S = \{$Bull (Up), Bear (Down), Stagnant$\}$ or $S = (-\infty, \infty)$.
- Credit Rating: $S = \{$AAA, AA, A, BBB, ..., Default$\}$ or $S = [0, 900]$.

The "Memoryless" Concept

The "Goldfish" Rule

- The process has no "memory" of the past.
- To predict where the system goes next, you only need to know where it is **right now**.
- Knowing how the system arrived at the current state provides no additional information about the future.

The Markovian Property

A stochastic process has the **Markovian Property** if its future behavior depends **only** on its current state, not on the path it took to get there.

A Simple Comparison

- **Markovian:** A board game like *Monopoly*. Your next position depends only on your current square and the roll of the dice. It doesn't matter if you've been in "Jail" three times already.

- **Non-Markovian:** Blackjack. The probability of the next card depends on every card that has already been dealt from the deck.

MC Definition

Discrete-Time Markov Chain

The stochastic process X is a discrete-time Markov chain with state space S if

$$P\{X_{n+1} = j | X_n = i, X_{n-1} = i_{n-1}, \ldots, X_1 = i_1, X_0 = i_0\} = P\{X_{n+1} = j | X_n = i\}$$

holds for all $i, j \in S$ and all $n = 0, 1, \cdots$

Transition Probability

The stochastic process X will transit from state i to state j with probability p_{ij}:

$$P\{X_{n+1} = j | X_n = i\} = p_{ij}$$

holds for all $i, j \in S$ and all $n = 0, 1, \cdots$

Markov Chain Types

Discrete-Time Markov Chains (DTMC)

In a DTMC, state changes happen at specific, separated "ticks" of a clock (e.g., $n = 0, 1, 2, \ldots$).
We are interested in the system's state after a certain number of steps.

Continuous-Time Markov Chains (CTMC)

In a CTMC, the system can change states at any random moment in time.
Instead of "steps," we look at the holding time (how long the system stays in a state)

DTMC: Transition Probability Matrix (P)

The probability of moving from state i to j is: $P\{X_{n+1} = j | X_n = i\} = p_{ij}$

Then, the (one-step) Transition Probability Matrix (P) is characterized by $P = \{p_{ij}\}$

Properties of P

- $p_{ij} \geq 0$ for all i, j
- $\sum_j p_{ij} = 1$ (The sum of probabilities leaving a state must be 1)

$$P = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ \vdots \\ n \end{array} \begin{array}{c} \begin{array}{cccccc} 0 & 1 & 2 & 3 & \cdots & n \end{array} \\ \begin{pmatrix} p_{00} & p_{01} & p_{02} & p_{03} & \cdots & p_{0n} \\ p_{10} & p_{11} & p_{12} & p_{13} & \cdots & p_{1n} \\ p_{20} & p_{21} & p_{22} & p_{23} & \cdots & p_{2n} \\ p_{30} & p_{31} & p_{32} & p_{33} & \cdots & p_{3n} \\ \cdots & \cdots & \cdots & \cdots & \ddots & \cdots \\ p_{n0} & p_{n1} & p_{n2} & p_{n3} & \cdots & p_{nn} \end{pmatrix} \end{array}$$

Example: A Simple Weather Model

The Scenario

- Let us assume that tomorrow's weather depends **only** on today's weather.
- We have two possible states: **Sunny** (0) and **Rainy** (1).

Transition Probabilities

- If it is **Sunny** today:
  - 80% chance it stays Sunny tomorrow.
  - 20% chance it rains tomorrow.
- If it is **Rainy** today:
  - 60% chance it stays Rainy tomorrow.
  - 40% chance it becomes Sunny tomorrow.

$$P = \begin{array}{c} \\ 0 \\ 1 \end{array} \begin{array}{c} \begin{array}{cc} 0 & 1 \end{array} \\ \begin{pmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{pmatrix} \end{array}$$

A Gambling Example

Consider a player starting with **$1** who plays a repetitive game:

- **Win:** Gains $1 with probability $p > 0$.
- **Loss:** Loses $1 with probability $1 - p > 0$.
- **Termination:** The game ends when the player:
  - Accumulates **$3**.
  - Goes **broke ($0)**.

$$P = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \end{array} \begin{array}{c} \begin{array}{cccc} 0 & 1 & 2 & 3 \end{array} \\ \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1-p & 0 & p & 0 \\ 0 & 1-p & 0 & p \\ 0 & 0 & 0 & 1 \end{pmatrix} \end{array}$$

n-step Transition Probabilities

The n-step transition probability $p_{ij}^{(n)}$ is the conditional probability that the system will be in state j after exactly n steps, given that it starts in state i in any time t.

$$p_{ij}^{(n)} = P\{X_{t+n} = j | X_t = i\}$$

$$\mathbf{P}^{(n)} = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ \vdots \\ n \end{array} \begin{array}{c} \begin{array}{cccccc} 0 & 1 & 2 & 3 & \cdots & n \end{array} \\ \begin{pmatrix} p_{00}^{(n)} & p_{01}^{(n)} & p_{02}^{(n)} & p_{03}^{(n)} & \cdots & p_{0n}^{(n)} \\ p_{10}^{(n)} & p_{11}^{(n)} & p_{12}^{(n)} & p_{13}^{(n)} & \cdots & p_{1n}^{(n)} \\ p_{20}^{(n)} & p_{21}^{(n)} & p_{22}^{(n)} & p_{23}^{(n)} & \cdots & p_{2n}^{(n)} \\ p_{30}^{(n)} & p_{31}^{(n)} & p_{32}^{(n)} & p_{33}^{(n)} & \cdots & p_{3n}^{(n)} \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ p_{n0}^{(n)} & p_{n1}^{(n)} & p_{n2}^{(n)} & p_{n3}^{(n)} & \cdots & p_{nn}^{(n)} \end{pmatrix} \end{array}$$

The Chapman-Kolmogorov Equations

For any i, j and $0 < m < n$, the n-step transition probability is:

$$p_{ij}^{(n)} = \sum_{k=0}^{M} p_{ik}^{(m)} p_{kj}^{(n-m)}$$

Intuition

To go from state i to state j in n steps, the process must be in some intermediate state k after m steps ($m < n$). We sum the probabilities of all possible intermediate paths through k.

For any n, the n-step transition matrix is the 1-step matrix raised to the n-th power.

$$\mathbf{P}^{(n)} = \mathbf{P}^n$$

The Weather Example

- 2-Step Transitions:

$$\mathbf{P}^{(2)} = \mathbf{P}^2 = \begin{bmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{bmatrix} \begin{bmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{bmatrix} = \begin{bmatrix} 0.72 & 0.28 \\ 0.56 & 0.44 \end{bmatrix}$$

- 3-Step Transitions:

$$\mathbf{P}^{(3)} = \mathbf{P}^3 = \begin{bmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{bmatrix} \begin{bmatrix} 0.72 & 0.28 \\ 0.56 & 0.44 \end{bmatrix} = \begin{bmatrix} 0.688 & 0.312 \\ 0.624 & 0.376 \end{bmatrix}$$

- 5-Step Transitions:

$$\mathbf{P}^{(6)} = \mathbf{P}^6 = \begin{bmatrix} 0.688 & 0.312 \\ 0.624 & 0.376 \end{bmatrix} \begin{bmatrix} 0.688 & 0.312 \\ 0.624 & 0.376 \end{bmatrix} = \begin{bmatrix} 0.66 & 0.33 \\ 0.66 & 0.33 \end{bmatrix}$$

Steady-State Probabilities

In the last example, after five steps, the two rows of P had identical enteries. This means that the probability of weather being in a particular state is independent of the state of the weather five days ago!

These probabilities (in either row) are called **Steady-State Probabilities** of the Markov chain.

$$\lim_{n \to \infty} P_{ij}^{(n)} = \pi_j$$

Steady-State (Limiting) Probabilities

We often use $\pi^{(0)}$ to refer to the initial probability distribution of the system at time t.

State Distribution After k Steps:

$$\pi^{(k)} = \pi^{(0)} P^k$$

Steady-State (Long-Term) Distribution:

$$\pi = \pi \mathbf{P} \quad \text{and} \quad \sum_j \pi_j = 1$$

Weather Example: Steady-State Probabilities

We use:

$$\pi = \pi \mathbf{P} \quad \text{and} \quad \sum_j \pi_j = 1$$

For our matrix $\mathbf{P} = \begin{bmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{bmatrix}$, we get:

$$\begin{bmatrix} \pi_0 & \pi_1 \end{bmatrix} = \begin{bmatrix} \pi_0 & \pi_1 \end{bmatrix} \begin{bmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{bmatrix} \Rightarrow \begin{cases} \pi_0 = 0.8\pi_0 + 0.4\pi_1 \\ \pi_1 = 0.2\pi_0 + 0.6\pi_1 \\ \pi_0 + \pi_1 = 1 \end{cases}$$

From these equations, we get:

$$\begin{cases} \pi_0 = \frac{2}{3} \\ \pi_1 = \frac{1}{3} \end{cases}$$

Classification of States

- State j is **accessible** from state i if $p_{ij}^{(n)} > 0$ for some $n \geq 0$. We use $i \to j$ to denote it
- If $i \to j$ and $j \to i$, then states i and j are said to **communicate**.
- Those states that can communicate with each other form a **Class**.

Irreducibility

A Markov chain with a single class (i.e., all states communicate) is irreducible.

Aperiodicity

A state is periodic if the chain can only return to it at fixed intervals (e.g., every 2 steps, every 3 steps). If the chain doesn't have these rigid cycles, it is **aperiodic**.

Classification of States

Absorbing State

- A state is absorbing if, after entering it, the process will never leave it again.
- A state i is absorbing if $p_{ii} = 1$.

Transient State

- State i is transient if, given we start in i, there is a non-zero probability we never return.
- If i is transient, there is a state j such that $i \to j$ but $j \nrightarrow i$.

Recurrent State

- State i is recurrent if we are guaranteed to return to i with probability 1.
- A state is recurrent if and only if it is not transient.

Properties of Markov Chains

Property	Simple Definition	If it FAILS...
Irreducible	Every state is reachable from every other state.	The system splits into separate, independent sub-systems.
Aperiodic	Transitions don't happen in a fixed, repeating "beat."	The system oscillates and never settles into a steady state.
Positive Recurrent	You always return to a state in a finite number of steps.	The system might "drift off" to infinity and never return.

The Ergodic Property

The Ergodic Property

If a Markov chain is **irreducible**, **aperiodic**, and **positive recurrent**, it is called **Ergodic**.

From these properties, we get the conclusion that a unique steady-state distribution π exists.

Classification of States: Example

$$P = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{ccccc} 0 & 1 & 2 & 3 & 4 \end{array} \\ \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0.2 & 0.8 & 0 & 0 & 0 \\ 0 & 0 & 0.4 & 0.6 & 0 \\ 0 & 0 & 0.7 & 0.3 & 0 \\ 0.1 & 0 & 0.2 & 0 & 0.7 \end{pmatrix} \end{array}$$

States

- Absorbing: 0
- Recutrrent: 2 and 3
- Transient: 1 and 4

Random Walk

A Markov Chain with transition probability P

$$P = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ \vdots \\ \vdots \\ N-1 \\ N \end{array} \begin{array}{c} \begin{array}{ccccccc} 0 & 1 & 2 & \cdots & \cdots & N-1 & N \end{array} \\ \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ q & 0 & p & 0 & 0 & 0 & 0 \\ 0 & q & 0 & p & 0 & 0 & 0 \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ 0 & 0 & 0 & 0 & q & 0 & p \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \end{array}$$

is called a Random Walk, where $p + q = 1$.

Random Walk: Biased vs. Unbiased

Problem Setup: Start at state i, terminal boundaries at 0 and N.

Unbiased (Fair Walk: $p = q = 0.5$)

- **Prob. of Success (P_i):**

$$P_i = \frac{i}{N}$$

- **Expected Duration ($E[T]$):**

$$E[T] = i(N - i)$$

Biased (Drift Walk: $p \neq q$)

- **Prob. of Success (P_i):**

$$P_i = \frac{1 - (q/p)^i}{1 - (q/p)^N}$$

- **Expected Duration ($E[T]$):**

$$E[T] = \frac{1}{p - q}\left[N(P_i) - i\right]$$

CTMC Definition

Continues-Time Markov Chain

A continuous-time stochastic process $\{X(t), t \geq 0\}$ has the Markov Property if, for all times $0 \leq s < t$ and states j, the conditional probability of the future state depends only on the present state

$$P\{X(t+s) = j \mid X(s) = i, X(u) = x(u) : 0 \leq u < s\} = P\{X(t+s) = j \mid X(s) = i\}$$

Transition Function

$P_{ij}(t)$ is the probability that a Markov chain, presently in state i, will be in state j after an additional time t:

$$P_{ij}(t) = P\{X(t+s) = j \mid X(s) = i\}$$

The Role of the Exponential Distribution

In a CTMC, the time T_i spent in state i must be independent of how long the process has already been in state i.

The Parameter q_i

The holding time T_i follows an **Exponential Distribution** with rate parameter q_i:

$$f_{T_i}(t) = q_i e^{-q_i t}, \quad t \geq 0$$

- q_i: The transition rate (frequency of events per unit time).
- $E[T_i] = 1/q_i$: The mean time spent in state i.

The Memoryless Property

The Exponential distribution is the **only** continuous distribution that has the memoryless property.
The probability of transitioning in the next t seconds is the same, whether you just arrived in the state or have been there for an hour.

Transitions and Rates in CTMC

Transition Rates

- q_{ij} **(Specific Rate):** The rate at which the process transitions from state i to state j ($i \neq j$).
- q_i **(Total Exit Rate):** The speed at which the process leaves state i, regardless of the destination.

The Fundamental Relationship

The total exit rate is the sum of all individual transition rates leaving state i:

$$q_i = \sum_{j \neq i} q_{ij}$$

Infinitesimal Generator Matrix Q

The Infinitesimal Generator Matrix is a square matrix that completely defines the dynamics of a Continuous-Time Markov Chain. Unlike a standard transition matrix that uses probabilities, the generator matrix uses **transition rates**.

The Logic

- Let q_{ij} be the rate of flowing **out** of i into j.
- Let $q_i = \sum_{j \neq i} q_{ij}$ be the **total rate** of leaving state i.
- We define the diagonal as $q_{ii} = -q_i$.

The Result

Summing across row i:

$$\sum_j q_{ij} = q_{ii} + \sum_{j \neq i} q_{ij} = -q_i + q_i = 0$$

Infinitesimal Generator Matrix

For a general Continuous-Time Markov Chain with $N+1$ states (indexed from 0 to N), the generator matrix Q represents the instantaneous transition rates between any two states.

$$q_{ii} = -\sum_{j \neq i} q_{ij} = -q_i$$

$$Q = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ \vdots \\ \vdots \\ N-1 \\ N \end{array} \begin{array}{c} \begin{array}{ccccccc} 0 & 1 & 2 & \cdots & \cdots & N-1 & N \end{array} \\ \begin{pmatrix} -q_0 & q_{01} & q_{02} & \cdots & \cdots & q_{0,N-1} & q_{0N} \\ q_{10} & -q_1 & q_{12} & \cdots & \cdots & q_{1,N-1} & q_{1N} \\ q_{20} & q_{21} & -q_2 & \cdots & \cdots & q_{2,N-1} & q_{2N} \\ \vdots & \vdots & \vdots & \ddots & & \vdots & \vdots \\ \vdots & \vdots & \vdots & & \ddots & \vdots & \vdots \\ q_{N-1,0} & q_{N-1,1} & q_{N-1,2} & \cdots & \cdots & -q_{N-1} & q_{N-1,N} \\ q_{N,0} & q_{N,1} & q_{N,2} & \cdots & \cdots & q_{N,N-1} & -q_N \end{pmatrix} \end{array}$$

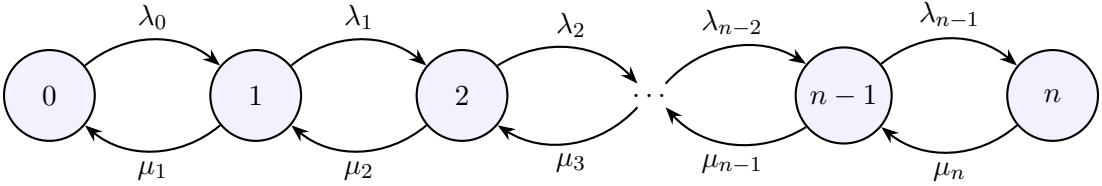
Birth and Death Processes

Definition: A CTMC is a birth and death process if the state space is $S = \{0, 1, \dots\}$ and $q_{ij} = 0$ if $|i - j| > 1$.

Transition Rates

When the process is in state i, the transition rates are defined as:

- $q_{i,i+1} = \lambda_i$ for $i = 0, 1, \dots$ (Birth Rates)
- $q_{i,i-1} = \mu_i$ for $i = 1, 2, \dots$ (Death Rates)



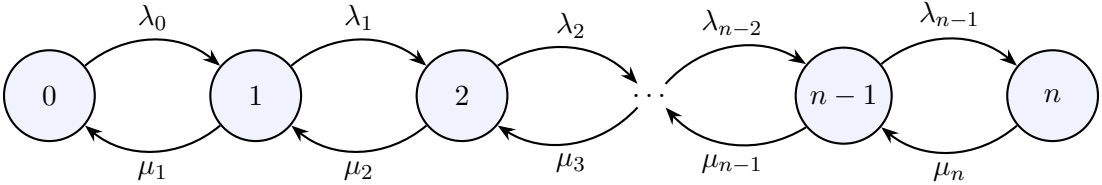
Birth and Death Processes: Generator Matrix

Jump Probabilities:

$$P_{i,i+1} = \frac{\lambda_i}{\lambda_i + \mu_i} \quad \text{and} \quad P_{i,i-1} = \frac{\mu_i}{\lambda_i + \mu_i}$$

$$Q = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ \vdots \\ i \\ \vdots \\ N \end{array} \begin{array}{c} \begin{array}{ccccccc} 0 & 1 & 2 & \cdots & i & \cdots & N \end{array} \\ \begin{pmatrix} -\lambda_0 & \lambda_0 & 0 & \cdots & 0 & \cdots & 0 \\ \mu_1 & -(\lambda_1+\mu_1) & \lambda_1 & \cdots & 0 & \cdots & 0 \\ 0 & \mu_2 & -(\lambda_2+\mu_2) & \lambda_2 & \cdots & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & \mu_i & -(\lambda_i+\mu_i) & \lambda_i & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 0 & \mu_N & -\mu_N \end{pmatrix} \end{array}$$

Birth-Death Process (0 to n)



Competing Exponential Variables

From state i, the process "chooses" the next transition:

- Probability of a Birth: $P_{i,i+1} = \frac{\lambda_i}{\lambda_i+\mu_i}$
- Probability of a Death: $P_{i,i-1} = \frac{\mu_i}{\lambda_i+\mu_i}$

The $M/M/s$ Queuing System

System Definition:

- **Arrivals:** Poisson process with constant rate λ.
- **Service:** s identical servers; service times are Exponential with rate μ.
- **Capacity:** Unlimited waiting spaces (infinite buffer).

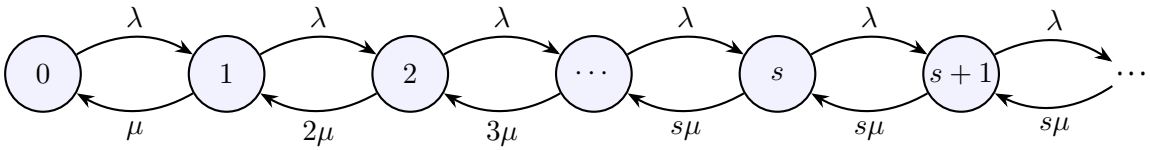
The Birth-Death Process: Let $X(t)$ be the number of customers in the system at time t. The state space is $S = \{0, 1, 2, \dots\}$.

Transition Rates

- **Birth Rates (Arrivals):** $\lambda_n = \lambda$ for all $n \geq 0$.
- **Death Rates (Service Completions):**

$$\mu_n = \begin{cases} n\mu & 1 \leq n \leq s; \\ s\mu & n > s. \end{cases}$$

The $M/M/s$ Queuing System



Queue Stability

To have a stable queue, we always need $\lambda \leq s\mu$.

Server utilization:

$$\rho = \frac{\lambda}{s\mu}$$

Queuing System Metrics

Metric	Symbol	Definition
Avg. customers (System)	L	Total items in queue + service
Avg. customers (Queue)	L_q	Items specifically waiting
Avg. wait time (System)	W	Total time spent in system
Avg. wait time (Queue)	W_q	Time spent before service starts

Little's Law: The Fundamental Link

$$L = \lambda W \quad \text{and} \quad L_q = \lambda W_q$$

Expected Number in System (L) and Queue (L_q)

$$L = \sum_{n=0}^{\infty} n \times \pi_n \quad \text{and} \quad L_q = \sum_{n=s}^{\infty} (n - s) \times \pi_n$$

Limiting Probabilities in CTMC

For an irreducible, positive recurrent CTMC, the limiting probabilities $\pi_j = \lim_{t\to\infty} P(X(t) = j)$ exist and are independent of the initial state.

The Equilibrium Equations

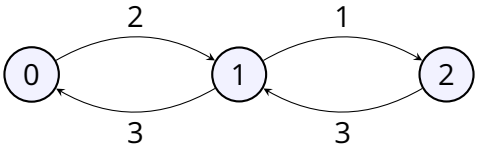
The vector π is the unique solution to:

1. $\pi Q = 0$ (Balance: Flow In = Flow Out)
2. $\sum_j \pi_j = 1$ (Normalization)

- π_j represents the **long-run proportion of time** spent in state j.
- $1/\pi_j$ is proportional to the **expected return time** to state j.

Example: Machine Repair Problem

Two machines ($\lambda = 1$), 1 repairman ($\mu = 3$). Let $X(t)$ be the number of broken machines.



Generator Matrix

$$Q = \begin{array}{c} \\ 0 \\ 1 \\ 2 \end{array} \begin{array}{c} \begin{array}{ccc} 0 & 1 & 2 \end{array} \\ \begin{pmatrix} -2 & 2 & 0 \\ 3 & -4 & 1 \\ 0 & 3 & -3 \end{pmatrix} \end{array}$$

Example: Machine Repair Problem

State	Rate Out	Rate In
0	$2\pi_0$	$3\pi_1$
1	$4\pi_1$	$2\pi_0 + 3\pi_2$
2	$3\pi_2$	$1\pi_1$

Balance Equations: Flow Out = Flow In

- State 0: $2\pi_0 = 3\pi_1 \implies \pi_1 = \frac{2}{3}\pi_0$
- State 2: $1\pi_1 = 3\pi_2 \implies \pi_2 = \frac{1}{3}\pi_1 = \frac{2}{9}\pi_0$
- Normalization: $\pi_0 + \pi_1 + \pi_2 = 1 \implies \pi_0(1 + \frac{2}{3} + \frac{2}{9}) = 1 \implies \pi_0(\frac{17}{9}) = 1$

$$\pi_0 = \frac{9}{17} = 0.53 \qquad \pi_1 = \frac{6}{17} = 0.35 \qquad \pi_2 = \frac{2}{17} = 0.12$$

Linking Dynamic Programming (DP) and Markov Chains

- **From Dynamic Programming:** We inherit **Optimization**. We seek a sequence of decisions that maximizes a long-term objective.
- **From Markov Chains:** We inherit the concept of **State Transitions**. The future depends only on the present state (*Markov Property*).

The Synthesis: Markov Decision Process (MDP)

In an MDP, the transition to state s' is no longer purely passive. It is a function of both the current state s and an **Action** a.

The Goal: Use DP techniques to find the best action for every possible state in a stochastic (random) environment.

MDP Definitions & Terminology

An MDP is formally defined by the 5-Tuple (S, A, P, R, γ):

- S **State Space:** The set of all possible situations (e.g., Robot location).
- A **Action Space:** The choices available (e.g., Move Left, Move Right).
- $P(s'|s,a)$ **Transition Probability:** The likelihood of landing in s' given state s and action a.
- $R(s,a)$ **Reward Function:** The immediate feedback for taking action a in state s.
- γ **Discount Factor:** A value $\in [0,1]$ weighting future vs. immediate rewards.

The Policy (π)

A strategy $\pi(s) \to a$ that specifies which action to take in each state to maximize the expected return.

Example: The Retailer's Dilemma

A retail manager must determine an ordering policy that maximizes long-term profit in the presence of uncertain demand.

- **The Sequential Decision:** Every Monday, we observe the stock and decide how much to order.
- **The Uncertainty:** Customer demand is stochastic (e.g., follows a Poisson distribution).
- **The Trade-off:** High stock ensures sales but increases holding costs; low stock saves money but risks stockouts.

Example: MDP Mathematical Formulation

We define the system as a tuple (S, A, P, R, γ):

States and Actions

- $s_t \in \{0, 1, \ldots, Max\}$: Inventory level at time t.
- $a_t \geq 0$: Quantity ordered at time t.

Reward Function $R(s, a)$

$$R(s, a) = \text{Revenue}(Sales) - \text{Cost}(Order) - \text{Cost}(Holding)$$

Example: The Bellman Equation

To find the optimal ordering policy π^*, we solve for the Value Function $V(s)$:

$$V^*(s) = \max_{a \geq 0} \left\{ R(s,a) + \gamma \sum_{s' \in S} P(s'|s,a) V^*(s') \right\}$$

- $\gamma \in [0,1)$: The discount factor (the time value of money).
- $P(s'|s,a)$: The probability that inventory s becomes s' after ordering a and experiencing random demand D, i.e.,

$$s' = \max(0, s + a - D)$$

The Bellman Optimality Equation

The goal is to find the **Value (V)** of being in a state. The value isn't just the immediate reward; it's the immediate reward plus the discounted sum of all future rewards you expect to get from that point onward.

The Equation

$$V(s) = \max_a \left(R(s,a) + \gamma \sum_{s' \in S} P(s' \mid s,a) V(s') \right)$$

Breaking it down:

- $\max_a$: We choose the action that gives the highest total result.
- $R(s,a)$: The **"Instant Gratification"** (Immediate Reward).
- $\gamma \sum P(s' \mid s,a) V(s')$: The **"Future Outlook."**
  - We multiply the probability of landing in a new state (P) by the value of that new state ($V(s')$).
  - We then discount it (γ) because a bird in the hand is worth two in the bush.

Summary of the Relationship

MC

Markov Chains provide the map (the probabilities of moving between states).

DP

Dynamic Programming provides the engine (the recursive math used to find the best path).

MDP

MDPs add the steering wheel (the actions and rewards).

Reinforcement Learning — Solving MDPs "In the Wild"

- If Markov Chains were the "map" and MDPs the "steering wheel," then Reinforcement Learning (RL) would be the driver learning to use them through trial and error.
- MDPs are solved with known probabilities (P) and the rewards (R). In the real world, we rarely have that "God view".
- Reinforcement Learning is the field of AI dedicated to solving MDPs when the environment is a black box.

Reinforcement Learning

- Reinforcement Learning (RL) is essentially solving an MDP where the **Transition Probabilities (P)** and **Reward Functions (R)** are unknown.
- In RL, the agent has no map. It must **interact** with the environment to learn through trial and error.

The RL Loop

The agent takes an **Action** (a), the environment provides a new **State** (s') and a **Reward** (r). The agent uses this experience to estimate the values that DP would have calculated.

From Bellman to Q-Learning

When we don't know $P(s'|s,a)$, we can't use the standard Bellman equation. Instead, we learn **Q-Values** (Quality), which represent the value of taking a specific action in a specific state.

The Q-Learning Update Rule

$$Q(s,a) \leftarrow Q(s,a) + \alpha[\underbrace{R + \gamma \max_{a'} Q(s',a')}_{\text{New Evidence}} - \underbrace{Q(s,a)}_{\text{Old Estimate}}]$$

The Logic:

- α **(Learning Rate):** Determines how quickly we overwrite old memories with new data.
- **Temporal Difference (TD) Error:** The difference between what we thought would happen and the actual reward received.

Methods for Solving MDPs

Exact Methods (Model-Based)

Requires $P(s'|s,a)$ and $R(s,a)$ to be known.

- **Value Iteration:** Iterative updates via the Bellman operator.
- **Policy Iteration:** Alternates between Evaluation and Improvement.
- **Linear Programming:** Solves the MDP as an optimization problem (e.g., using Gurobi).

Learning Methods (Model-Free)

Used when environment dynamics are unknown or complex.

- **Q-Learning:** Learns $Q(s,a)$ through experience and TD-error.
- **Monte Carlo:** Learns from sampled episodes of complete transitions.
- **Deep RL:** Uses Neural Networks to approximate V or Q functions.

Synthesis: Descriptive to Prescriptive to Adaptive

Markov Chains (Descriptive)

How does the system behave?
Example: Predicting ER wait times using $\pi = \pi P$.

MDPs (Prescriptive):

How **should** the system behave?
Example: Finding the optimal inventory policy via Bellman.

Reinforcement Learning (Adaptive):

How do we decide when the "rules" are unknown?
Example: Using Q-Learning to manage supply chains in volatile markets.

Decision making under uncertainty is the core of modern Business Analytics.